

UNIT IV

TRANSACTION CONTROL COMMANDS and PL/SQL Concepts

Commit :

COMMIT command in SQL is used to save all the transaction-related changes permanently to the disk. Whenever DDL commands such as INSERT, UPDATE and DELETE are used, the changes made by these commands are permanent only after closing the current session. So before closing the session, one can easily roll back the changes made by the DDL commands. Hence, if we want the changes to be saved permanently to the disk without closing the session, we will use the commit command.

Syntax:

1. **COMMIT;**

Example:

We will select an existing database, i.e., school.

Insert 10 records in it.

OUT PUT

```
mysql> COMMIT;  
Query OK, 0 rows affected (0.03 sec)
```

SAVEPOINT

We can divide the database operations into parts. For example, we can consider all the insert related queries that we will execute consecutively as one part of the transaction and the delete command as the other part of the transaction. Using the SAVEPOINT command in SQL, we can save these different parts of the same transaction using different names. **For example**, we can save all the insert related queries with the savepoint named INS. To save all the insert related queries in one savepoint, we have to execute the

SAVEPOINT query followed by the savepoint name after finishing the insert command execution.

Syntax:

1. SAVEPOINT savepoint_name; \

```
mysql> SAVEPOINT Insertion;  
mysql> SAVEPOINT Insertion;  
Query OK, 0 rows affected (0.00 sec)
```

ROLLBACK

While carrying a transaction, we must create savepoints to save different parts of the transaction. According to the user's changing requirements, he/she can roll back the transaction to different savepoints. *Consider a scenario:* We have initiated a transaction followed by the table creation and record insertion into the table. After inserting records, we have created a savepoint INS. Then we executed a delete query, but later we thought that mistakenly we had removed the useful record. Therefore in such situations, we have an option of rolling back our transaction. In this case, we have to roll back our transaction using the *ROLLBACK* command to the savepoint INS, which we have created before executing the DELETE query.

Syntax:

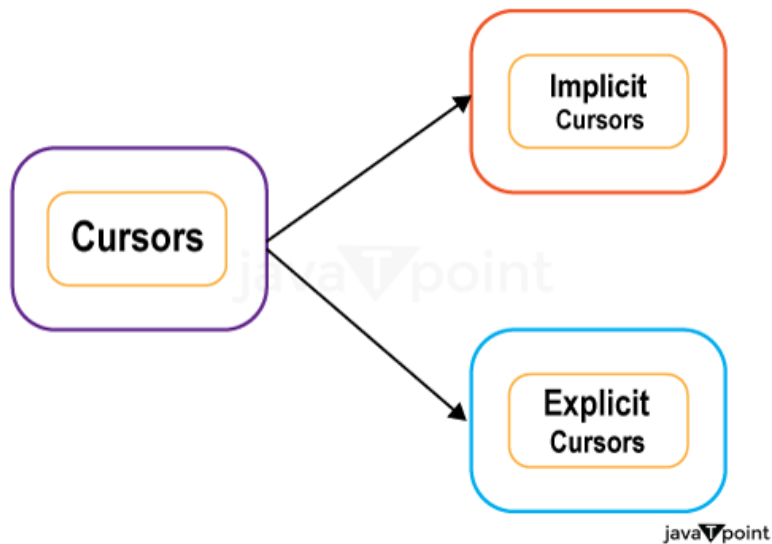
1. **ROLLBACK TO** savepoint_name;

CURSORS:-

PL/SQL Cursor

When an SQL statement is processed, Oracle creates a memory area known as context area. A cursor is a pointer to this context area. It contains all information needed for processing the statement. In PL/SQL, the context area is controlled by Cursor. A cursor contains information on a select statement and the rows of data accessed by it.

A cursor is used to referred to a program to fetch and process the rows returned by the SQL statement, one at a time. There are two types of cursors:



1) PL/SQL Implicit Cursors

The implicit cursors are automatically generated by Oracle while an SQL statement is executed, if you don't use an explicit cursor for the statement.

This type of cursor handles many of the cursor related functions such as:

- Save the area in memory
- Populating this area with relevant data
- Processing the data in memory area
- Free the memory area when processing is complete

These are created by default to process the statements when DML statements like INSERT, UPDATE, DELETE etc. are executed.

Oracle provides some attributes known as Implicit cursor's attributes to check the status of DML operations. Some of them are: %FOUND, %NOTFOUND, %ROWCOUNT and %ISOPEN.

For example: When you execute the SQL statements like INSERT, UPDATE, DELETE then the cursor attributes tell whether any rows are affected and how many have been affected. If you run a SELECT INTO statement in PL/SQL block, the implicit cursor attribute can be used to find out whether any row has been returned by the SELECT statement. It will return an error if there no data is selected.

The following table specifies the status of the cursor with each of its attribute.

--	--

%FOUND	Its return value is TRUE if DML statements like INSERT, DELETE and UPDATE affect at least one row or more rows or a SELECT INTO statement returned one or more rows. Otherwise it returns FALSE.
%NOTFOUND	Its return value is TRUE if DML statements like INSERT, DELETE and UPDATE affect no row, or a SELECT INTO statement return no rows. Otherwise it returns FALSE. It is a just opposite of %FOUND.
%ISOPEN	It always returns FALSE for implicit cursors, because the SQL cursor is automatically closed after executing its associated SQL statements.
%ROWCOUNT	It returns the number of rows affected by DML statements like INSERT, DELETE, and UPDATE or returned by a SELECT INTO statement.

PL/SQL Implicit Cursor Example

Create customers table and have records:

1	Ramesh	23	Allahabad	20000
2	Suresh	22	Kanpur	22000
3	Mahesh	24	Ghaziabad	24000
4	Chandan	25	Noida	26000
5	Alex	21	Paris	28000
6	Sunita	20	Delhi	30000

Let's execute the following program to update the table and increase salary of each customer by 5000. Here, SQL%ROWCOUNT attribute is used to determine the number of rows affected:

Create procedure:

```
DECLARE
    total_rows number(2);
BEGIN
    UPDATE customers
    SET salary = salary + 5000;
    IF sql%notfound THEN
        dbms_output.put_line('no customers updated');
    ELSIF sql%found THEN
        total_rows := sql%rowcount;
        dbms_output.put_line( total_rows || ' customers updated ');
    END IF;
END;
/
```

Output:

*6 customers updated
PL/SQL procedure successfully completed.*

2) PL/SQL Explicit Cursors

The Explicit cursors are defined by the programmers to gain more control over the context area. These cursors should be defined in the declaration section of the PL/SQL block. It is created on a SELECT statement which returns more than one row.

Following is the syntax to create an explicit cursor:

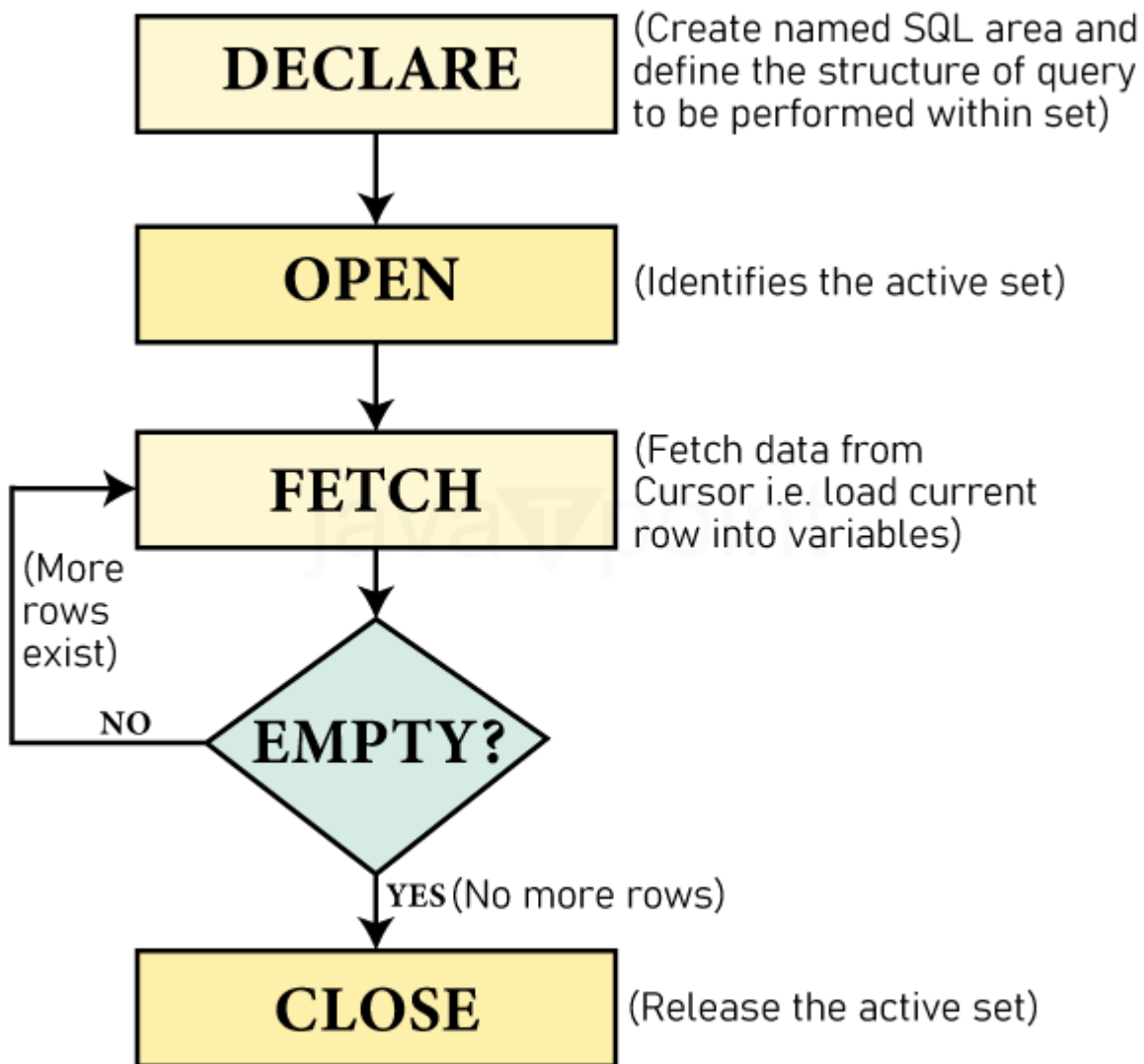
Syntax of explicit cursor

Following is the syntax to create an explicit cursor:

1. **CURSOR** cursor_name **IS** select_statement;;

Steps:

You must follow these steps while working with an explicit cursor.



java^{point}

1. Declare the cursor to initialize in the memory.
2. Open the cursor to allocate memory.
3. Fetch the cursor to retrieve data.
4. Close the cursor to release allocated memory.

1) Declare the cursor:

It defines the cursor with a name and the associated SELECT statement. Every cursor must be declared before use in the declaration segment of the PL/SQL block. Declaring a cursor means giving it a name and specifying the SELECT statement with which the cursor is associated. At this time there is no memory allocation. Declaration segment of PL/SQL block may declare more than one cursors at a time but of different names.

Syntax for explicit cursor declaration

1. **CURSOR** name IS
2. **SELECT** statement;

2) Open the cursor:

It is used to allocate memory for the cursor and make it easy to fetch the rows returned by the SQL statements into it. After the declaration, the cursor is opened with OPEN statement to process the rows in the cursor.

While opening the cursor, the actual memory allocation to the cursor takes place which is crucial for processing information. The SELECT statement related with the cursor is parsed. It also locates the active data set i.e. rows of all included that are declared to the cursor and are created when the cursor is opened. When the OPEN statement is performed, rows are not retrieved from the active data set. Instead, the FETECH statement retrieves the rows. It also places a pointer just before the first row in the active set.

Syntax for cursor open:

1. **OPEN** cursor_name;

Trigger in SQL

In this article, you will learn about the trigger and its implementation with examples.

A **Trigger** in Structured Query Language is a set of procedural statements which are executed automatically when there is any response to certain events on the particular table in the database. Triggers are used to protect the data integrity in the database.

In Structured Query Language, triggers are called only either before or after the below events:

1. **INSERT Event:** This event is called when the new row is entered in the table.
2. **UPDATE Event:** This event is called when the existing record is changed or modified in the table.
3. **DELETE Event:** This event is called when the existing record is removed from the table.

Types of Triggers in SQL

Following are the six types of triggers in SQL:

- | | | |
|---|---------------|----------------|
| 1. AFTER | INSERT | Trigger |
| This trigger is invoked after the insertion of data in the table. | | |
| 2. AFTER | UPDATE | Trigger |
| This trigger is invoked in SQL after the modification of the data in the table. | | |
| 3. AFTER | DELETE | Trigger |
| This trigger is invoked after deleting the data from the table. | | |

- | | | | |
|----|---|---------------|----------------|
| 4. | BEFORE | INSERT | Trigger |
| | This trigger is invoked before the inserting the record in the table. | | |
| 5. | BEFORE | UPDATE | Trigger |
| | This trigger is invoked before the updating the record in the table. | | |
| 6. | BEFORE | DELETE | Trigger |
| | This trigger is invoked before deleting the record from the table. | | |

Syntax of Trigger in SQL

1. **CREATE TRIGGER** Trigger_Name
2. [BEFORE | **AFTER**] [**Insert** | **Update** | **Delete**]
3. **ON** [Table_Name]
4. [**FOR EACH ROW** | **FOR EACH COLUMN**]
5. **AS**
6. **Set of** SQL Statement

In the trigger syntax, firstly, we have to define the name of the trigger after the CREATE TRIGGER keyword. After that, we have to define the BEFORE or AFTER keyword with anyone event.

Then, we define the name of that table on which trigger is to occur.

After the table name, we have to define the row-level or statement-level trigger.

And, at last, we have to write the SQL statements which perform actions on the occurring of event.

Example of Trigger in SQL

To understand the concept of trigger in SQL, first, we have to create the table on which trigger is to be executed.

The following query creates the **Student_Trigger** table in the SQL database:

```
CREATE TABLE Student_Trigger
(
  Student_RollNo INT NOT NULL PRIMARY KEY,
  Student_FirstName Varchar (100),
  Student_EnglishMarks INT,
  Student_PhysicsMarks INT,
  Student_ChemistryMarks INT,
  Student_MathsMarks INT,
  Student_TotalMarks INT,
  Student_Percentage );
```

The following query shows the structure of the **Student_Trigger** table:

```
DESC Student_Trigger;
```

Output:

Student_RollNo	INT	NO	PRI	NULL	
Student_FirstName	Varchar(100)	YES		NULL	
Student_EnglishMarks	INT	YES		NULL	
Student_PhysicsMarks	INT	YES		NULL	
Student_ChemistryMarks	INT	YES		NULL	
Student_MathsMarks	INT	YES		NULL	
Student_TotalMarks	INT	YES		NULL	
Student_Percentage	INT	YES		NULL	

Advantages of Triggers in SQL

Following are the three main advantages of triggers in Structured Query Language:

1. SQL provides an alternate way for maintaining the data and referential integrity in the tables.
2. Triggers helps in executing the scheduled tasks because they are called automatically.
3. They catch the errors in the database layer of various businesses.
4. They allow the database users to validate values before inserting and updating.

Disadvantages of Triggers in SQL

Following are the main disadvantages of triggers in Structured Query Language:

1. They are not compiled.
2. It is not possible to find and debug the errors in triggers.
3. If we use the complex code in the trigger, it makes the application run slower.
4. Trigger increases the high load on the database system.